

Data Structure

03. 연결 리스트

3주차: 연결 리스트

? 오늘의 핵심 질문

? 질문 1

연결 리스트는 왜 중간 삽입과 삭제에 유리할까?

? 질문 2

반대로, 왜 원하는 위치를 곧바로 찾는(인덱스 접근) 데는 느릴까?

? 질문 3

Python 개발자는 연결 리스트를 거의 직접 구현해 쓰지 않는데도 왜 반드시 배워야 할까?

목차 INDEX

01 연결 리스트

- 연결 리스트
- 노드
- Head & Tail
- 메모리 구조

02 연결 리스트 동작

- 순회
- 삽입
- 삭제
- 시간복잡도

03 연결 리스트 종류

- 단일 연결 리스트
- 이중 연결 리스트

04 기타

- 파이썬에서 연결 리스트를 안쓰는 이유
- 연결 리스트를 배우는 이유

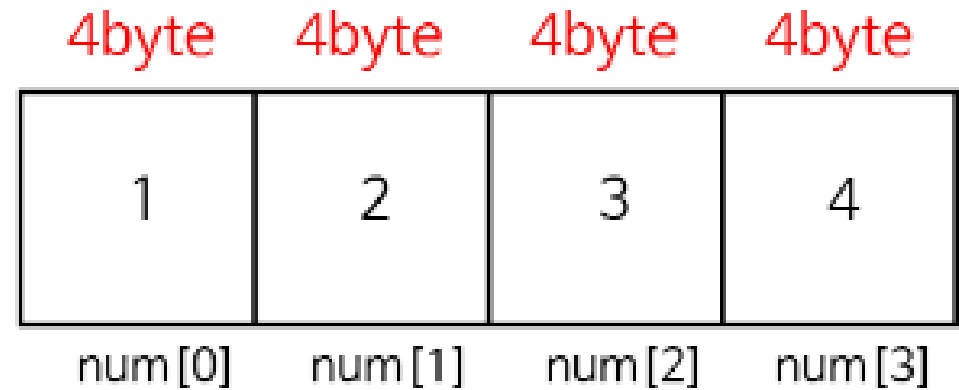
01

연결 리스트

Linked Lists

Review: Array

- 이렇게 생긴게 배열이었음
- 배열의 원소들은 메모리 상에서 물리적으로 나란히 저장됨 (주소가 바로 옆임)
- 이러한 구조는 읽거나 접근(인덱스) 속도는 압도적이지만, 중간에 데이터를 넣거나 지울 때 뒤의 원소들을 통째로 옮겨야 한다는 문제가 있음



Review: Array

[10] [20] [30] [40] [50]

20과 30 사이에 25를 넣으려면?

-> 30, 40, 50을 뒤로 한 칸씩 이동 후 빈 자리에 25를 넣으면 됨

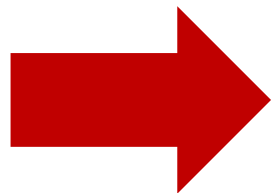
=> 근데 이렇게 하면 데이터를 이동시키는 데 시간이 걸릴듯

새로운 아이디어

```
[ 10 | next ] —> [ 20 | next ] —> [ 30 | next ]
```

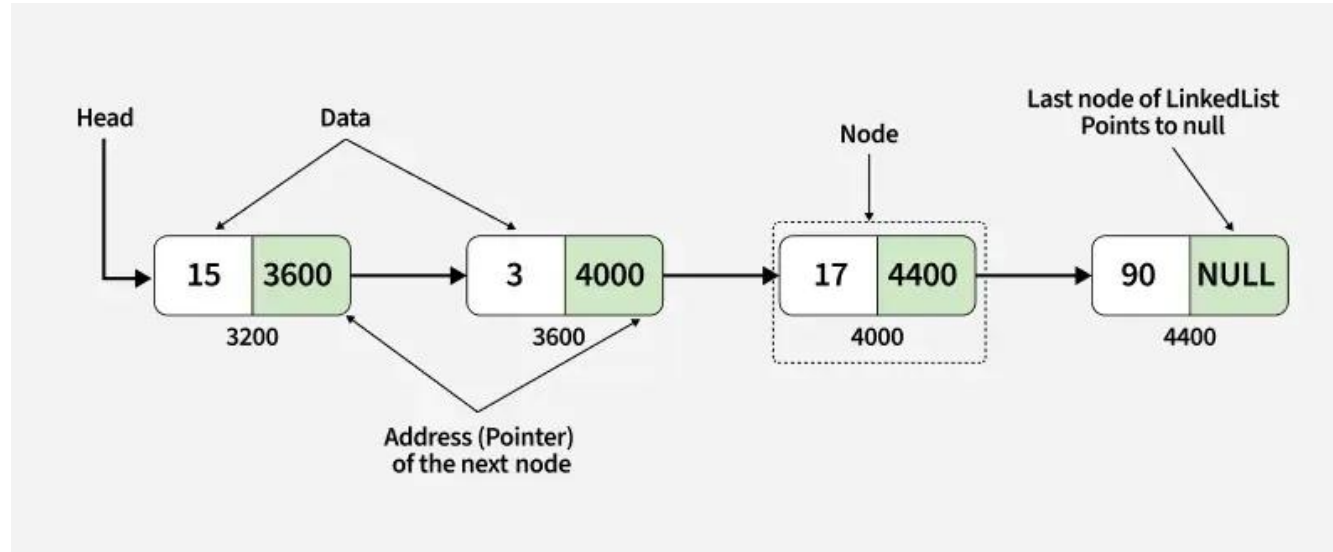
굳이 붙여서 저장해야 하나?

서로 다음 위치만 기억하고 있으면 되지 않을까?



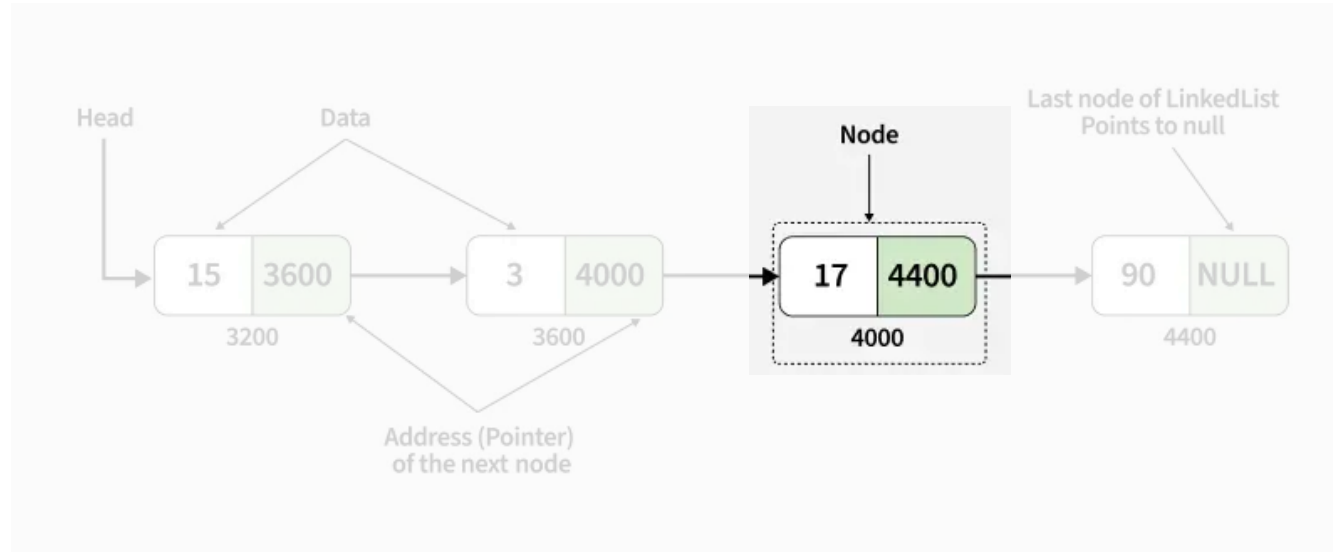
Linked Lists

Linked Lists



연결 리스트는 이렇게 생김
데이터와 그 다음 칸의 주소를 기억하면
리스트가 길어져도 문제 없을거같음

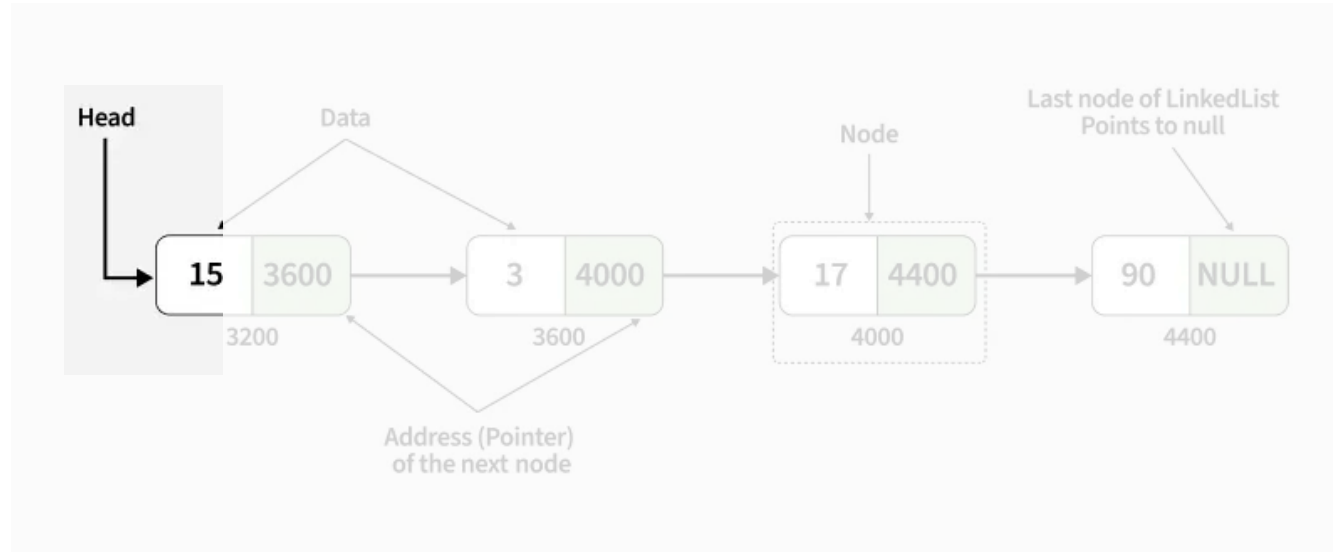
Linked Lists: Node



노드는 데이터와 다음 칸 주소를 통틀어 말한 것임
밑의 4000은 노드 자체의 주소임

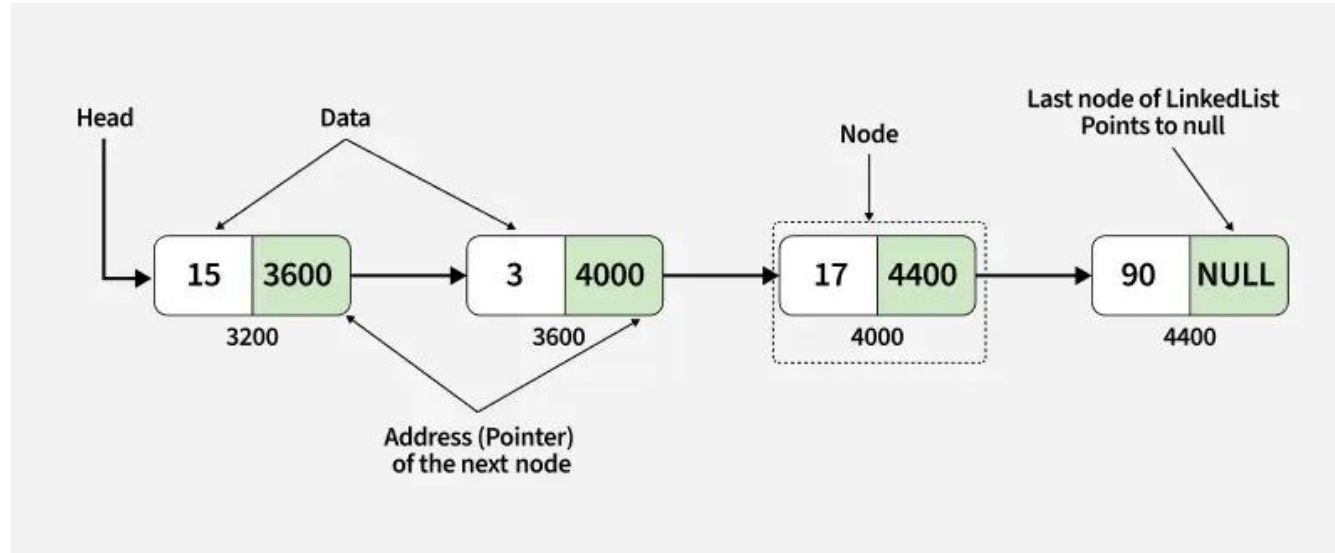
노드에 저장할 다음 칸 주소는 다음 노드의 주소를 저장하면 됨
보통 노드에 있는 데이터는 Data, 다음 칸 주소는 Next라고 부름

Linked Lists: Head & Tail



헤드는 연결 리스트에서 최초 시작점을 의미함
테일은 연결 리스트의 끝점을 의미함 (append 할때 필요)
Head & Tail은 노드가 아니라, 시작과 끝 위치를 담은 참조 변수임

Singly Linked Lists



단일 연결 리스트는 말 그대로 단일 방향으로 연결된 리스트임
원하는 인덱스로 가려면 반드시 Head부터 시작해
차례대로 다음 노드를 따라가야 함

마지막 노드의 Next는 항상 None(NULL)을 가리켜 리스트의 끝임을 표시

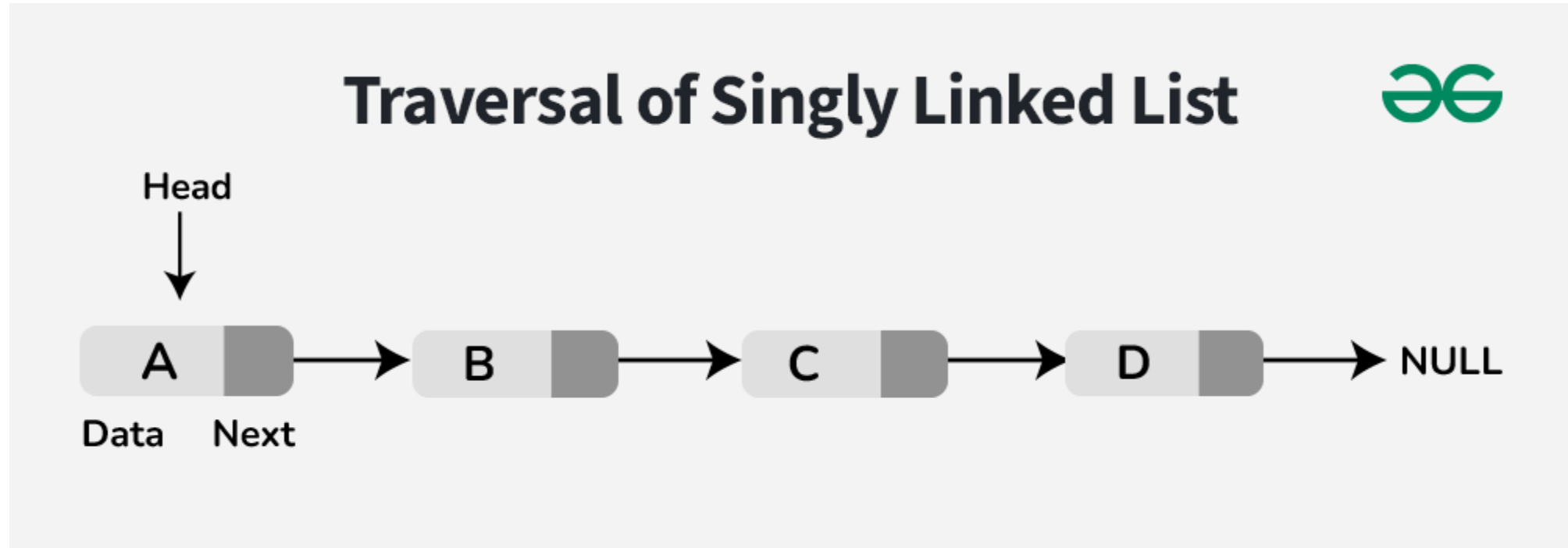
02

연결 리스트의 동작

Linked List Operations

Traversal: 순회

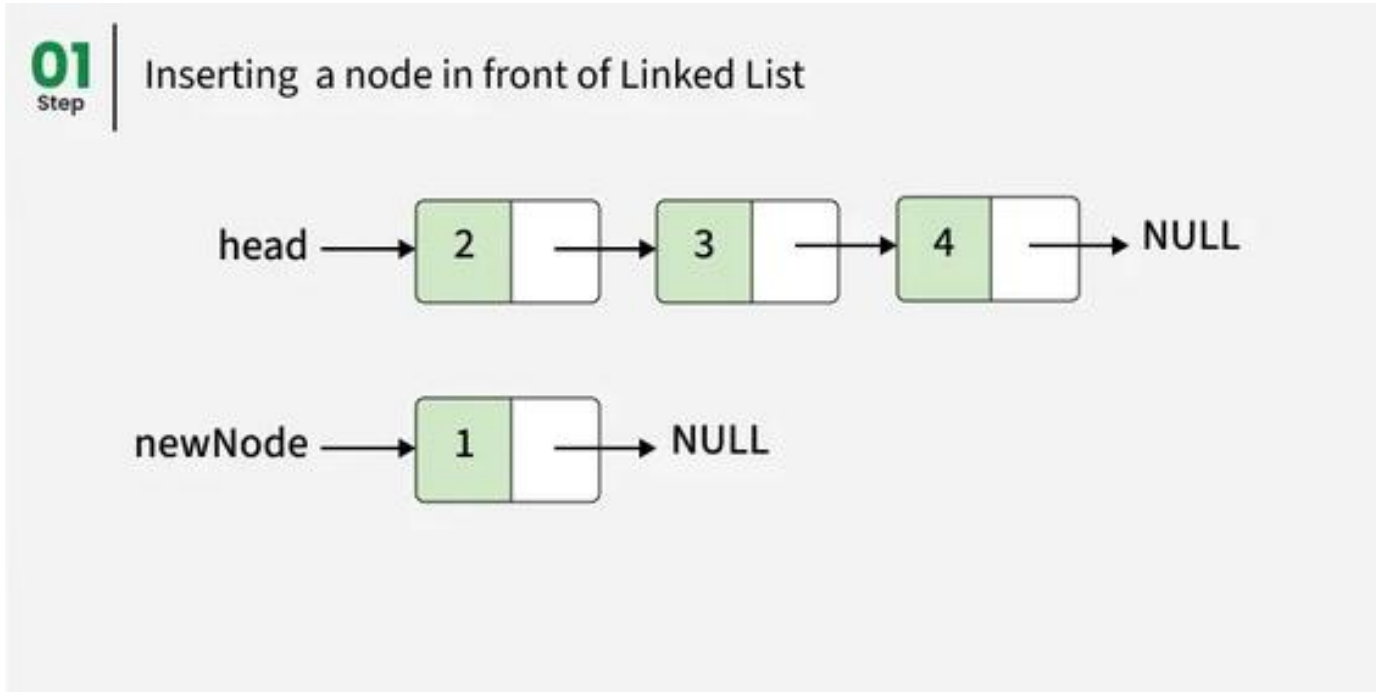
시간복잡도: $O(N)$



(이미지 클릭 시 gif로 보기 가능)

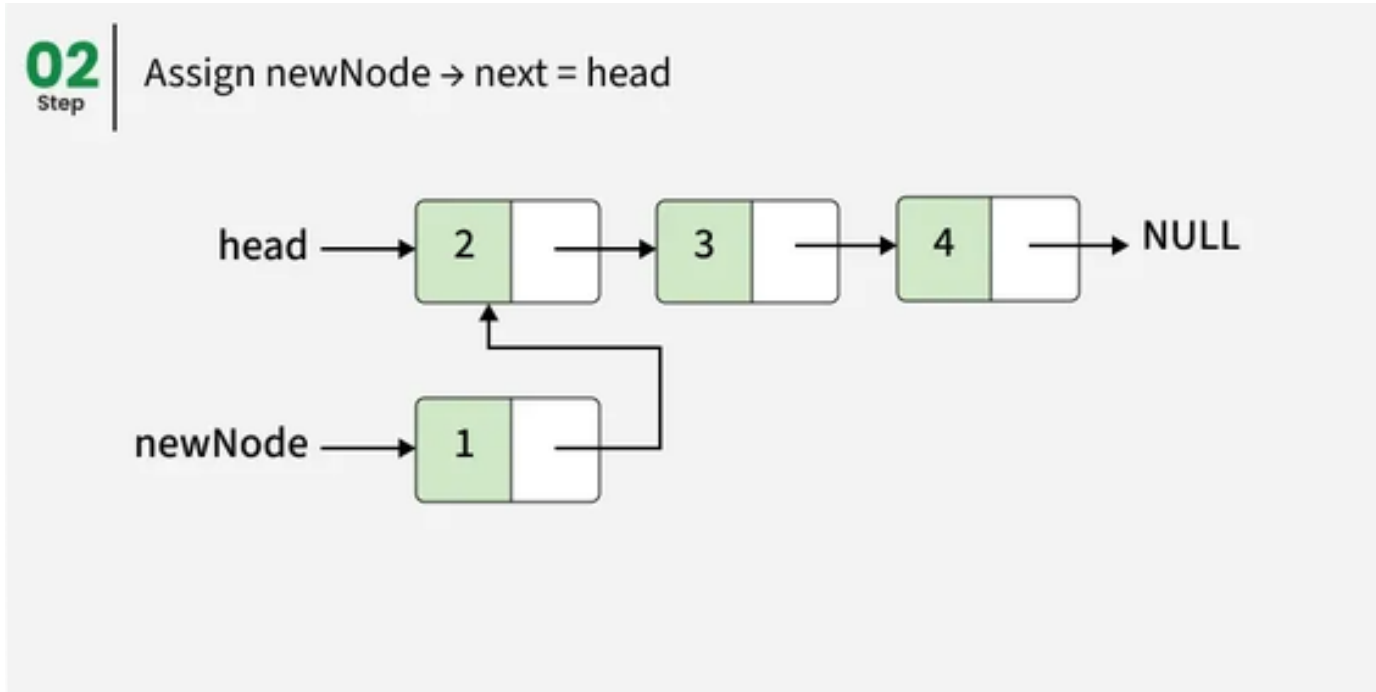
Head에서 시작해 각 노드의 Next를 따라
원하는 Data가 나올 때 까지 이동

Push Front: Head 앞 삽입



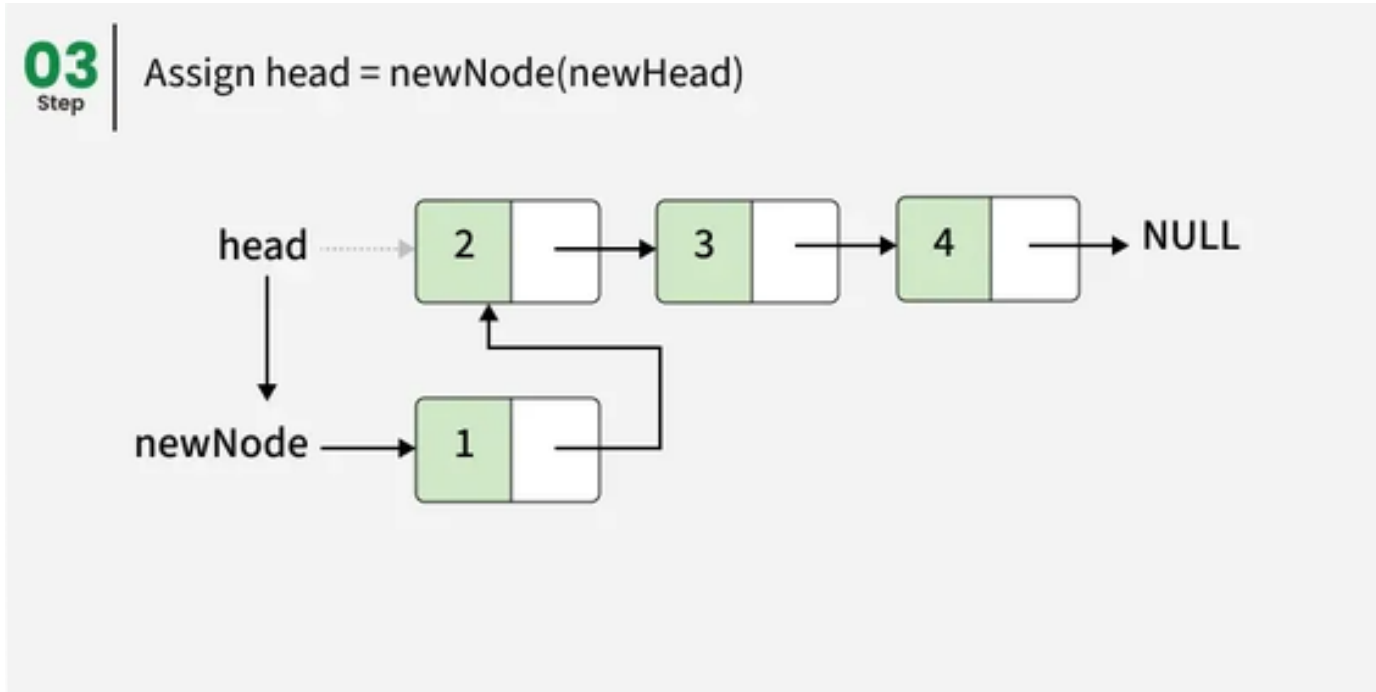
새로운 노드 (newNode) 준비
노드가 1개이므로 Next는 무조건 None을 가리켜야 함

Push Front: Head 앞 삽입



이후 newNode의 Next 주소에 Head가 가리키는 노드의 주소를 넣음
(newNode가 2 노드를 가리키게)

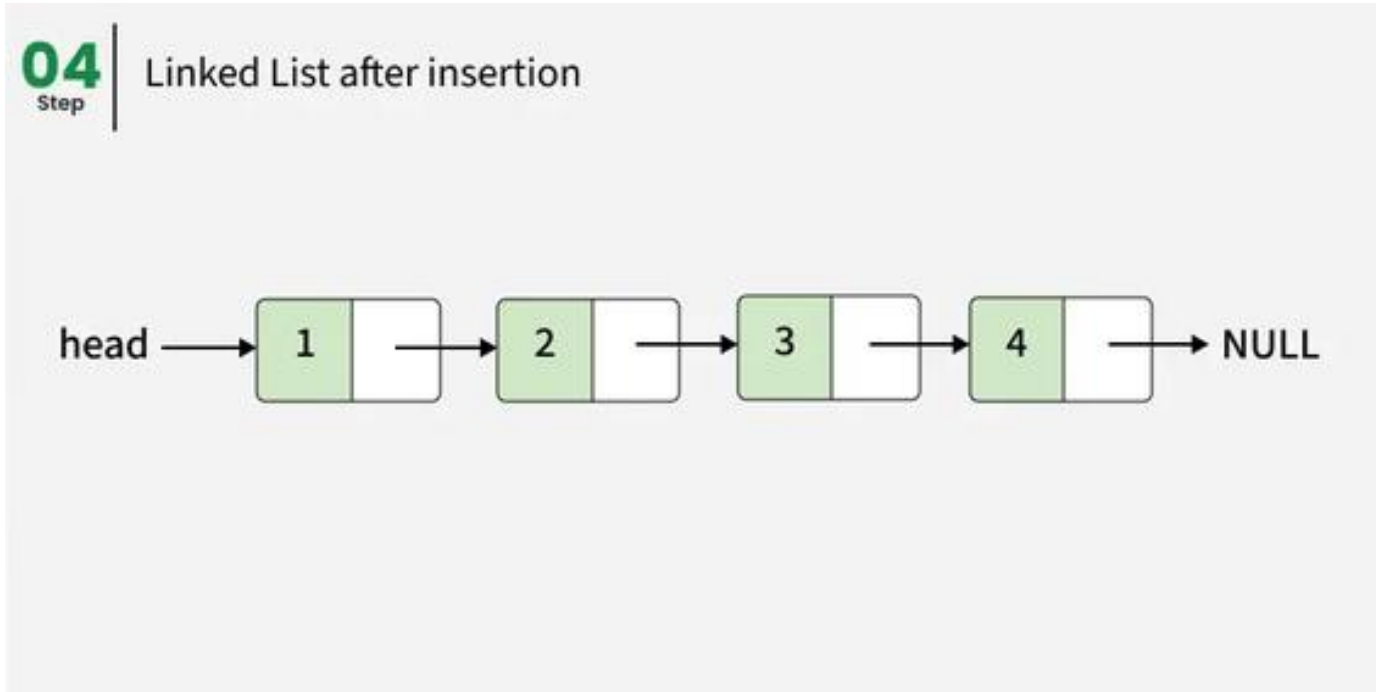
Push Front: Head 앞 삽입



그런 다음 Head가 newNode를 가리키도록 바꿈

Push Front: Head 앞 삽입

시간복잡도: $O(1)$



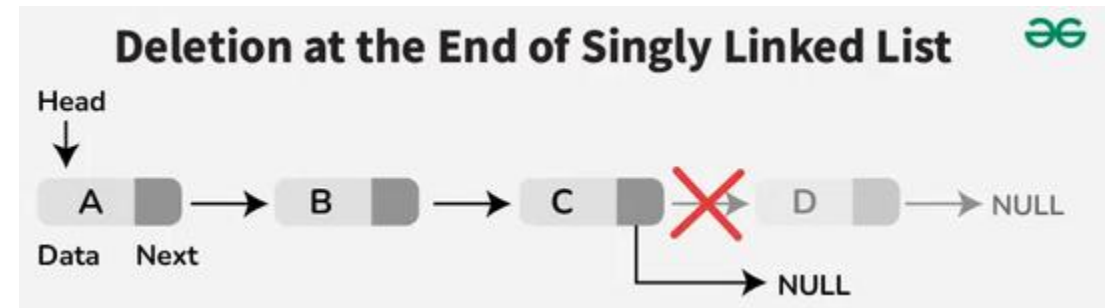
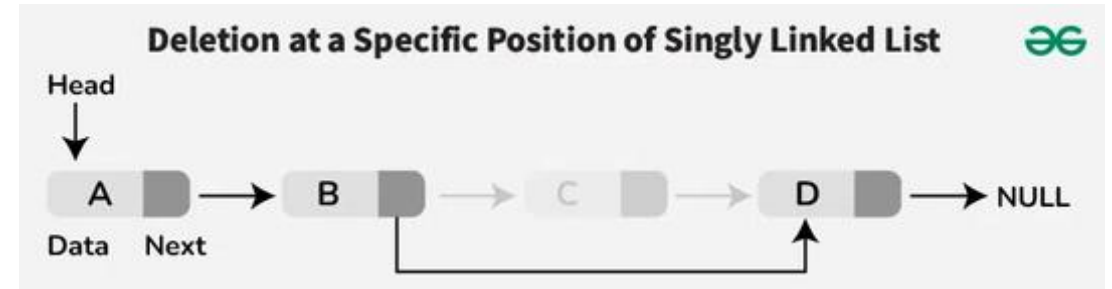
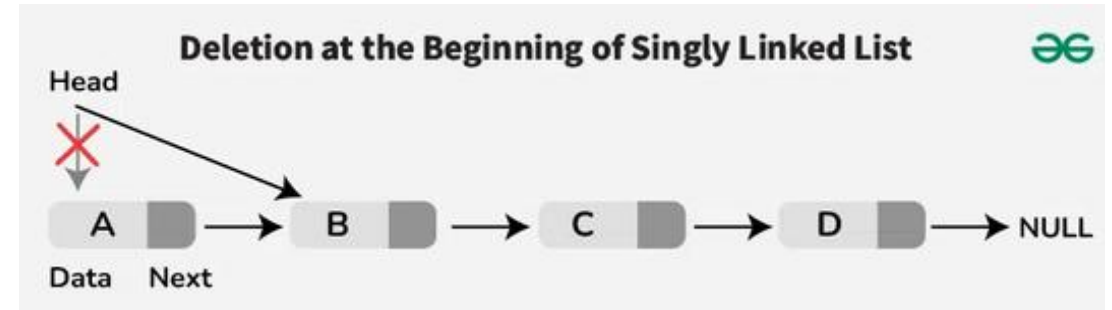
그렇게 하면 Head 앞에 1 노드 삽입 완료

리스트 중간에 삽입할 때에도 같은 과정을 거치면 됨

Deletion: 삭제

- 삽입 과정과 마찬가지로 각 노드의 Next를 조정하여 노드를 없앨 수 있음
- 삭제할 노드의 위치를 알고 있다면 $O(1)$ 의 복잡도를 가지는 작업임

시간복잡도: $O(1)$



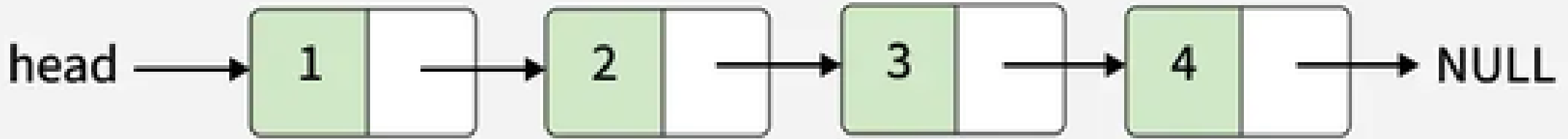
03

연결 리스트의 종류

Types Linked Lists

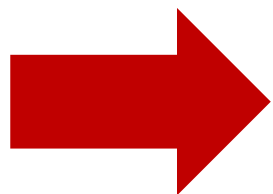
03 연결 리스트의 종류 Types of Linked List

사실 그대로 쓰기엔 좀 불편함



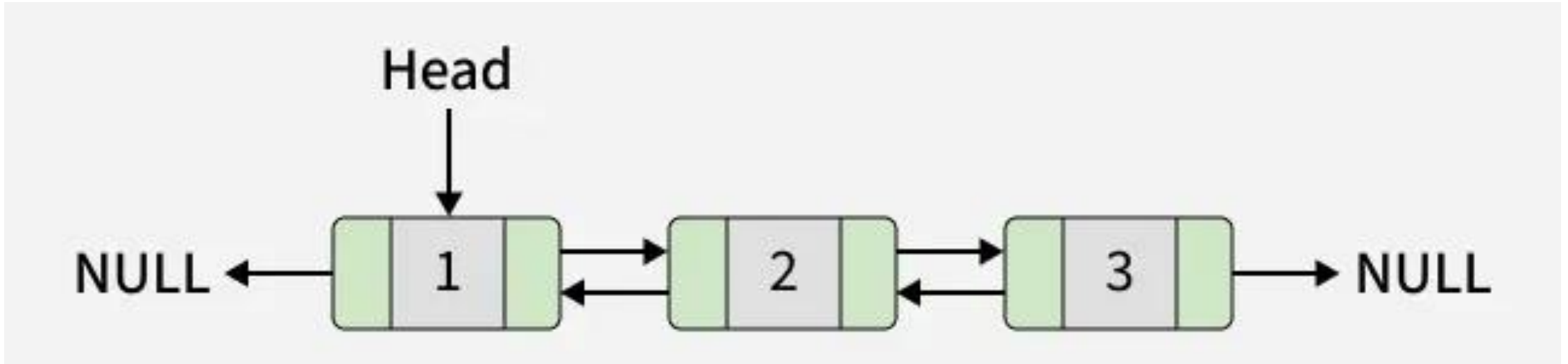
단일 연결 리스트 구조는 복잡도 면에서 상당히 괜찮은 구조임

그러나 현재 노드에서 이전 노드로 돌아갈 방법이 없음



화살표를 양방향으로 두면 될듯?

Doubly Linked List



각 노드에 Next 와 더불어 Prev를 추가함

Next는 다음 노드의 주소, Prev는 이전 노드의 주소를 저장함

다만 Prev를 따로 저장하므로 메모리를 약간 더 쓰긴 함

04

기타

Miscellaneous

Python에서 Linked List를 잘 안쓰는 이유

1. 파이썬의 리스트 자료형 자체가
이미 매우 강력하게 최적화된 Array라서
2. 파이썬에서 노드를 하나하나 생성하면
부가정보가 많아 메모리랑 속도 측면에서 손해라서
3. 안써도 웬만한 문제 다 list, set, dict 등으로 풀 수 있어서



그럼 안배워도 되는거 아니가요?

그래도 알아야 하는 이유

1. Tree, Graph 등의 기초라서
2. Hash Table에서도 나와서
3. Reference, Pointer 개념을 이해하는데 좋아서
4. 실무 수준 시스템을 짜려면 이게 필요해서
5. 기술면접에 많이 나와서